# Anitoscript (.ascript)

For GAMELAB AnitoTracer use

# Parsing and Transpiling

# What Is Automatic

The ANTLR lexer and parser C++ files are generated from the grammar files during the CMake build. In `CMakeLists.txt`, a custom command runs the ANTLR JAR with Java, and declares both `.g4` files and the JAR as dependencies. Generated files go into the build directory, under `src/Scripting/generated`; they are not handwritten source files to edit.

So, after changing a grammar file, **building the project regenerates the ANTLR C++ parser automatically**. The ANTLR command does not normally need to be manually run. Java and the configured ANTLR JAR must be available.

# What Is Handwritten

ANTLR's generated parser only recognizes the syntax and creates a parse tree. It does not decide what the language means or generate DLL code.

- `AnitoScriptLexer.g4` defines tokens such as keywords, operators and literals.
- `AnitoScriptParser.g4` defines the language's syntax rules: declarations, statements, expressions and precedence.
- `ScriptParser.cpp` runs the generated lexer/parser, captures syntax errors, and extracts component descriptors and exposed fields/default values.
- `ScriptAnalyzer.cpp` checks names and types and emits C++ for component bodies during the same traversal. It is both the semantic checker and the main component-code generator, rather than a separate typed-AST pass.
- `ScriptTranspiler.cpp` assembles the generated component classes into a module translation unit, including the DLL entry table and factories. It generally does not need changes for an ordinary new language expression or statement.
- `AnitoScriptSDK.hpp` defines the functions and types generated scripts can use to call into the engine.

The editor uses `ScriptParser::Parse()` directly; it does not run the `scriptc` executable. That parser/analyzer path produces each component's C++ source, and `BuildModuleSource()` assembles it into `Scripts.gen.cpp`.

# Adding a Feature

The work depends on whether the new feature changes syntax:

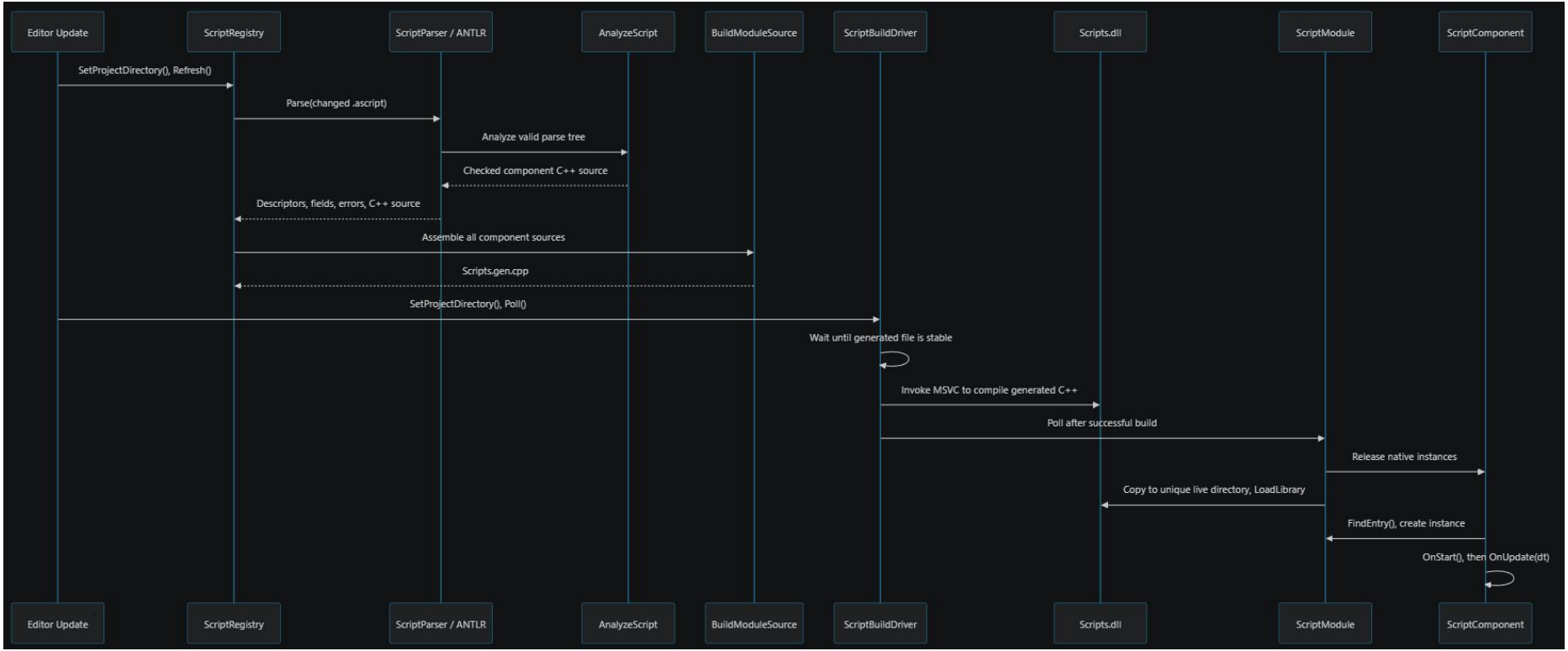
1. **Syntax change:** Update the appropriate `.g4` rule or rules. For new words or symbols, update the lexer grammar too. Pay attention to token conflicts and expression precedence.
2. **Parser integration:** Build once to regenerate ANTLR. Then update `ScriptParser.cpp` if the feature adds information the editor or descriptor needs. Inspect the generated parser context/accessor names if you need to consume a new grammar rule.
3. **Meaning and generated code:** Update `ScriptAnalyzer.cpp` to validate the feature and emit equivalent C++. If it calls a new engine operation, extend the SDK and host implementation too.
4. **Build and exercise it:** Build through CMake Tools, try valid and invalid examples with `anito_scriptc`, and verify the generated C++ compiles. Then test through the editor, where changed `.ascript` files flow into generated C++, a DLL build, and the reload path.

For example, adding `clamp(x, low, high)` does **not** need a grammar change: calls already use generic `name(args)` syntax. You would add its accepted argument types and return type to the analyzer, then emit the corresponding C++ expression. A new syntax form such as `for item in collection` **does** require a grammar change, plus semantic handling and a C++ translation in the analyzer.

# Rule of thumb

**ANTLR automates parser regeneration; it does not automate language-feature implementation.** A grammar edit is often only the first step. The build and editor pipeline then picks up the changes automatically once the handwritten parser/analyzer behavior supports them.

# Tracing


Editor Update
ScriptRegistry
ScriptParser / ANTLR
AnalyzeScript
BuildModuleSource
ScriptBuildDriver
Scripts.dll
ScriptModule
ScriptComponent
SetProjectDirectory(), Refresh()
Parse(changed .ascript)
Analyze valid parse tree
Checked component C++ source
Descriptors, fields, errors, C++ source
Assemble all component sources
Scripts.gen.cpp
SetProjectDirectory(), Poll()
Wait until generated file is stable
Invoke MSVC to compile generated C++
Poll after successful build
Release native instances
Copy to unique live directory, LoadLibrary
FindEntry(), create instance
OnStart(), then OnUpdate(dt)
Editor Update
ScriptRegistry
ScriptParser / ANTLR
AnalyzeScript
BuildModuleSource
ScriptBuildDriver
Scripts.dll
ScriptModule
ScriptComponent

# Startup and project selection

1. `AnitoTracer_App::InitManagers()` calls `ScriptRegistry::Initialize()`, which registers `ScriptComponent` with the engine's serialization type registry.
2. Each app update gets the active project directory and calls `ScriptRegistry::SetProjectDirectory()`, `ScriptRegistry::Refresh()`, `ScriptBuildDriver::SetProjectDirectory()` / `Poll()`, and `ScriptModule::SetModulePath()` / `Poll()`.
3. The runtime DLL path is `<project>/Library/Scripts/Scripts.dll`.

# Discover and parse scripts

1. `ScriptRegistry::Refresh()` scans `<project>/Scripts/` recursively for `.ascript` files. It throttles scans to about 500 ms and reparses a file when its last-write time changes.
2. `ScriptParser::Parse()` feeds the source to the generated ANTLR lexer and parser. Its error listener records syntax errors with file, line and column.
3. It extracts each `component` declaration into a `ScriptDescriptor`: component name, exposed `@field` declarations and defaults, event handlers, methods, and source path.

# Check semantics and produce component C++

1. On syntactically valid input, `AnalyzeScript()` checks names, types, expressions, calls, handlers, and assignment rules while emitting each component's C++ class.
2. Exposed fields become references into host-owned storage. Unexposed fields become ordinary class members.
    a. Generated method bodies receive `#line` directives, allowing C++ compiler diagnostics to refer back to `.ascript` source locations.
3. Semantic errors go into the same script error collection as syntax errors.

# Assemble the generated module

1. When files change, `ScriptRegistry::WriteGeneratedSource()` gathers the component C++ and calls `BuildModuleSource()`.
2. The module source includes the script SDK, all generated classes, field-type arrays, factory/destructor callbacks, and the exported `AnitoScript_GetModule()` entry point.
3. It writes `<project>/Library/Scripts/Scripts.gen.cpp` only when scripts are error-free and the generated content differs from the existing file. If scripts have errors, it keeps the previous generated file.

# Build the DLL

1. `ScriptBuildDriver` watches `Scripts.gen.cpp`. It waits about 700 ms after the file's timestamp stabilizes, then starts a background MSVC build.
2. Its compiler path and SDK include path come from CMake configuration. It initializes the MSVC environment using `vcvars64.bat`, then invokes `cl.exe` to compile and link the generated source as a DLL.
3. Output is `<project>/Library/Scripts/Scripts.dll`; compiler output is also written to `Scripts.build.log`.
4. On successful completion, the driver asks `ScriptModule` to poll. The editor's regular update-loop polling also detects the new DLL timestamp.

# Unload and reload

1. When `ScriptModule::Poll()` detects a changed DLL, it calls `ReleaseAllInstances()` before unloading the old module. That releases each script instance through the DLL's destroy callback while its code is still mapped.
2. It copies the DLL and its PDB to a unique `<project>/Library/Scripts/live/<n>/` directory, then calls `LoadLibraryW()` and finds `AnitoScript_GetModule()` with `GetProcAddress()`.
3. It checks the SDK ABI version, stores the module entry table, and increments the module generation. The unique copy keeps the build output free for the linker to overwrite.

# Create and run script instances

1. `ScriptComponent::OnUpdate()` calls `Refresh()` and `EnsureInstance()`.
2. `EnsureInstance()` finds the script descriptor, verifies that the DLL's field count and types match, gathers pointers to the component's serialized field values, and passes them with the owner pointer and host API in an `anito::ScriptContext`.
3. The DLL's factory creates the generated class. The component calls `OnStart()` once for that instance, then calls `OnUpdate(deltaTime)` on subsequent updates.
4. Generated code uses the SDK's host function table for operations such as logging, position access and rotation. This avoids linking script modules against the engine's internal registries.

# Questions

# What decides whether a word is a keyword, identifier, or type?

1. ANTLR's lexer classifies the text first. In `AnitoScriptLexer.g4`
2. The parser then allows those tokens in particular places.
3. Primitive type tokens are accepted by the grammar's `type` rule.
4. So **ANTLR identifies the spelling and parses its role; the analyzer enforces the language's meaning and restrictions.**

# How are syntax errors handled?

- The grammar defines what syntax is valid, and ANTLR's generated lexer/parser detects violations.
- `ScriptParser.cpp` installs an error listener that records the file, line, and column. ANTLR attempts to recover, so the parser can still return partial declarations for the editor.

# How would you implement user-defined types?

- The grammar already accepts an identifier in a type position, but ScriptAnalyzer.cpp currently rejects it as unknown.
- User-defined types for transpiling to .cpp is kinda hard. BUT to do this, this is the first steps:
    - Extend the analyzer’s type model and C++ emitter to resolve the new types;
    - validate member access and construction;
    - and emit the generated structs BEFORE component classes.

# Summary and Key files

# Editor Behavior and Current Scope

- The Add Component popup gets available script names from `ScriptRegistry`. Binding a script creates host-owned fields from its descriptor; the Inspector draws those serialized fields.
- The Inspector's script error dropdown shows syntax and semantic errors. C++ compiler output is currently in the console/build log rather than that dropdown.
- The supported runtime callbacks are `Start()` and `Update(dt)`. The SDK currently exposes a small host API rather than general engine access.
- The CMake build has succeeded, and the handwritten DLL hot reload was previously confirmed working. The new automatic `.ascript` → generated C++ → DLL → runtime reload loop still needs an end-to-end runtime test.

# Key Files

- `AnitoTracer_App.cpp`: coordinates registry, builder and module polling.
- `ScriptRegistry.cpp`: file discovery, parse cache, error storage and generated-source writing.
- `ScriptParser.cpp`: ANTLR parsing and descriptor extraction.
- `ScriptAnalyzer.cpp`: semantic checks and component-class generation.
- `ScriptTranspiler.cpp`: builds the exported DLL translation unit.
- `ScriptBuildDriver.cpp`: debounced asynchronous MSVC build.
- `ScriptModule.cpp`: DLL copy, load, validation, unload and reload.
- `ScriptComponent.cpp`: field storage, native-instance lifecycle and per-frame calls.
- `AnitoScriptSDK.hpp`: stable host/module ABI.